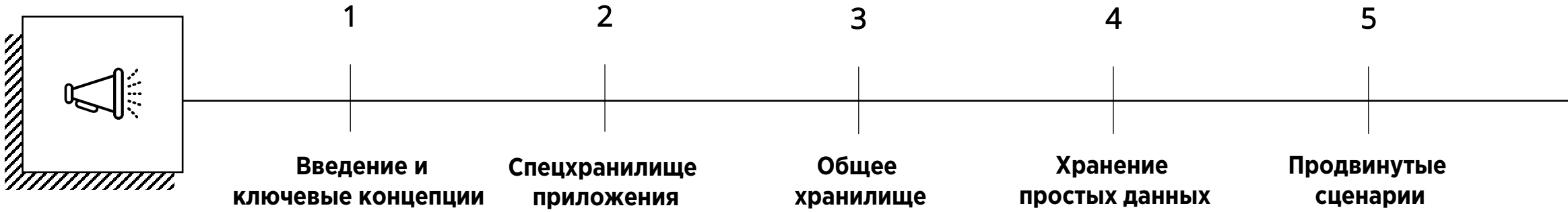
Файлы и данные приложения

Содержание лекции



Введение и ключевые концепции

Варианты хранения данных в Android

Файловые хранилища:

- Внутреннее (App-specific): Приватные файлы приложения. Только ваше приложение имеет доступ.
- Внешнее (Shared Storage): Общие файлы (фото, документы), доступные всем приложениям.

Структурированные данные:

- Настройки (Preferences): Хранение простых данных в формате «ключ-значение».
- Базы данных (Room): Хранение больших объемов структурированных данных.

Варианты хранения данных в Android

Представьте, что хранилище телефона — это квартира. Внутреннее хранилище — это ваш личный сейф. Общее — это гостиная, куда могут заходить гости (другие приложения)

Сравнительная таблица типов хранилищ

Тип данных	Где используется?	Доступ другим приложениям?	Удаляется с приложением?
Файлы приложения	Внутренняя/внешняя память (папка приложения)	Нет	Да
Медиа-контент	Общее хранилище (Фото, Видео, Аудио)	Да (с разрешением)	Нет
Документы	Общее хранилище (Загрузки, Документы)	Да (через системный выбор)	Нет
Настройки	Внутренняя память (приватно)	Нет	Да
База данных	Внутренняя память (приватно)	Нет	Да

Как выбрать правильное хранилище?

Эти данные должны быть доступны только моему приложению?
➡️ Да: Используйте Спецхранилище приложения (внутреннее или внешнее).

Нужно ли, чтобы данные остались после удаления приложения?
➡️ Да: Используйте Общее хранилище (MediaStore API).

Это простые настройки в формате «ключ-значение»?
➡️ Да: Используйте Preferences DataStore.

Это сложные, структурированные данные для поиска и фильтрации?
➡️ Да: Используйте Базу данных (Room).

Физическое расположение: Internal vs External

Internal (Внутреннее):
- Всегда доступно и надежно.
- Приватно для вашего приложения.
- Обычно имеет меньший объем.
- Идеально для критически важных данных.

External (Внешнее):
- Может быть недоступно (например, SD-карта).
- Подходит для больших объемов данных (кэш, медиа).
- Включает как папки приложения, так и общее хранилище.

ВАЖНО: Никогда не используйте жестко прописанные пути к файлам (/sdcard/...). Всегда получайте пути через методы API!

Эволюция доступа к хранилищу: Scoped Storage

Старый подход (до Android 10):

- Приложение запрашивало READ/WRITE_EXTERNAL_STORAGE.
- Получало доступ ко всему внешнему хранилищу.
- Это было небезопасно и приводило к «свалке» файлов.

Новый подход (Android 10+): Scoped Storage

- Приложения получают ограниченный доступ («область видимости»).
- По умолчанию: доступ только к своей папке и медиафайлам, которые само создало.
- Для доступа к чужим медиафайлам: нужно разрешение READ_EXTERNAL_STORAGE.
- Полный доступ (редкое исключение): разрешение MANAGE_EXTERNAL_STORAGE для файловых менеджеров.

Спецхранилище приложения

Спецхранилище: приватные данные приложения

Что это? Область памяти, предназначенная только для вашего приложения.

Главное свойство: Все файлы в этом хранилище удаляются при удалении приложения.

Два вида расположения:
- Внутреннее (Internal): Более надежное и безопасное.
- Внешнее (External): Для объемных, но не критичных файлов (например, кэш).

Примечание. Для дополнительной защиты файлов приложений используйте библиотеку безопасности, входящую в состав Android Jetpack, для шифрования этих файлов в состоянии покоя. Ключ шифрования зависит от вашего приложения.

Работа с ВНУТРЕННИМ хранилищем

Постоянные файлы (не удаляются системой):
- Используются для данных, которые должны существовать всегда.
- Как получить доступ: context.filesDir

Кэшированные файлы (могут быть удалены системой):
- Для временных данных, которые можно загрузить заново.
- Как получить доступ: context.cacheDir

Внутреннее хранилище: Базовые операции

Путь к папке:

```
context.filesDir (Объект File)
```

Запись текста в файл:

```
val filename = "my_app_data.txt"
val content = "This is important data."
context.openFileOutput(filename, Context.MODE_PRIVATE).use {
outputStream ->
    outputStream.write(content.toByteArray())
}
```

Внутреннее хранилище: Базовые операции

Чтение текста из файла:

```
context.openFileInput(filename).bufferedReader().useLines { lines
->
    val content = lines.joinToString("\n")
    Log.d("FileReader", content)
}
```

Получение списка файлов:

```
val fileList: Array<String> = context.fileList()
```

Внутреннее хранилище: Работа с Кэшем

Путь к папке:

```
context.cacheDir
```

Создание временного файла:

```
val tempFile = File.createTempFile("avatar_", ".jpg",
context.cacheDir)
Log.d("Cache", "Temp file path: ${tempFile.absolutePath}")
```

Внутреннее хранилище: Работа с Кэшем

Очистка кэша:

- Простой способ: Удалить все файлы из cacheDir:

```
cacheFile.delete()

context.deleteFile(cacheFileName)
```

- Пример удаления файла:

```
val fileToDelete = File(context.cacheDir, "old_data.tmp")
if (fileToDelete.exists()) {
    fileToDelete.delete()
}
```

Как безопасно поделиться файлом? FileProvider

Проблема: Файлы во внутреннем хранилище приватны. Как передать их другому приложению (например, для открытия PDF в читалке)?

Неправильное решение: Пытаться изменить права доступа к файлу. Это небезопасно и не работает на новых версиях Android.

Правильное решение: FileProvider

1. Генерирует специальный, временный content:// URI.
2. Предоставляет другому приложению временный доступ на чтение (или запись) только к этому конкретному файлу.
3. Полностью контролируется вами и является стандартом безопасности.

Работа с ВНЕШНИМ спецхранилищем

Когда использовать? Для больших, но не критичных данных, когда не хватает внутреннего хранилища.
Важная особенность: Хранилище может быть недоступно (например, SD-карта извлечена). Всегда проверяйте доступность перед использованием!

```
fun isExternalStorageWritable(): Boolean {
    return Environment.getExternalStorageState() ==
Environment.MEDIA_MOUNTED
}
```

Доступ к папкам (не требует разрешений):

- Постоянные файлы: context.getExternalFilesDir(null)
- Кэш: context.externalCacheDir

Внешнее спецхранилище: Пример с медиа

Проверка доступности (обязательно!):

```
if (Environment.getExternalStorageState() !=
Environment.MEDIA_MOUNTED) {
    // Хранилище недоступно, выходим
    return
}
```

Внешнее спецхранилище: Пример с медиа

Создание папки для изображений:

```
// Получаем папку Pictures внутри папки нашего приложения на
внешнем хранилище
val picturesDir =
context.getExternalFilesDir(Environment.DIRECTORY_PICTURES)

// Создаем файл в этой папке
val newImage = File(picturesDir, "my_photo.jpg")
```

Важно:

- getExternalFilesDir(TYPE) не требует разрешений.
- Файлы здесь удаляются вместе с приложением.

Управление свободным местом

Проблема: У пользователя может быть мало свободного места на устройстве.
Стратегия работы:

1. Не «копите» мусор: Регулярно очищайте свой кэш (cacheDir, externalCacheDir).
2. Перед сохранением большого файла: Можно попытаться записать его и обработать IOException, если места не хватило. Это самый простой способ.
3. Если места нет: Вы можете попросить пользователя освободить его, запустив системный диалог:

```
val intent = Intent(Settings.ACTION_MANAGE_STORAGE)
startActivity(intent)
```

Intent ACTION_CLEAR_APP_CACHE для очистки кэша всех приложений — очень агрессивное действие, которое влияет на батарею

Общее хранилище

Общее хранилище и MediaStore API

Что это? Стандартные папки на устройстве для медиа и документов (Фото, Музыка, Загрузки и т.д.).

Главное свойство: Файлы здесь не удаляются вместе с приложением и доступны другим приложениям (с разрешения).

Как с ним работать? MediaStore API — это единая точка входа.

- Это не прямой доступ к файлам, а работа через специальный «менеджер контента» — ContentResolver.
- Он предоставляет доступ к упорядоченным коллекциям: Images, Video, Audio, Downloads.

Разрешения для работы с Общим хранилищем

Чтобы СОЗДАТЬ медиафайл:

- Разрешения не нужны. Ваше приложение автоматически получает доступ к созданным им файлам.

Чтобы ЧИТАТЬ медиафайлы, созданные ДРУГИМИ приложениями:

- Нужно разрешение: <uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />

Чтобы ИЗМЕНЯТЬ/УДАЛЯТЬ медиафайлы, созданные ДРУГИми приложениями:

- Прямого разрешения нет. При попытке действия вы ловите исключение и просите у пользователя разрешение на изменение конкретного файла. (Подробнее — дальше).

Для файловых менеджеров:

- Существует специальное разрешение MANAGE_EXTERNAL_STORAGE, но Google Play разрешает его только для узкого круга приложений.

Практика: Чтение файлов (Query)

Задача: Найти все изображения на устройстве.

Практика: Чтение файлов (Query)

```
// 1. Определяем, какие колонки нам нужны
val projection = arrayOf(MediaStore.Images.Media._ID,
MediaStore.Images.Media.DISPLAY_NAME)
// 2. Делаем запрос через ContentResolver
context.contentResolver.query(
    MediaStore.Images.Media.EXTERNAL_CONTENT_URI, // URI коллекции картинок
    projection, // Какие колонки вернуть
    null, null, null // Условия выборки и сортировки
)?.use { cursor ->
    // 3. Итерируемся по результатам
    while (cursor.moveToNext()) {
        val name =
cursor.getString(cursor.getColumnIndexOrThrow(MediaStore.Images.Media.DISPLAY_NAME
))
        Log.d("MediaStore", "Found image: $name")
    }
}
```

Практика: Чтение файлов (Query)

Критически важно: Этот код должен выполняться в фоновом потоке!

Практика: Создание файла (Insert)

Задача: Сохранить изображение в галерею в папку «MyPictures».

Практика: Создание файла (Insert)

```
val values = ContentValues().apply {
    put(MediaStore.Images.Media.DISPLAY_NAME, "my_vacation_photo.jpg")
    put(MediaStore.Images.Media.MIME_TYPE, "image/jpeg")
    // Указываем, куда положить файл
    put(MediaStore.Images.Media.RELATIVE_PATH,
Environment.DIRECTORY_PICTURES + "/MyPictures")
    // Флаг для эксклюзивного доступа на время записи
    put(MediaStore.Images.Media.IS_PENDING, 1)
}

val resolver = context.contentResolver
val uri = resolver.insert(MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
values)
```

Практика: Создание файла (Insert)

```
uri?.let {
    // Записываем данные в файл
    resolver.openOutputStream(it).use { stream ->
        /* ... запись bitmap ... */
    }

    // Снимаем флаг, делая файл видимым для всех
    values.clear()
    values.put(MediaStore.Images.Media.IS_PENDING, 0)
    resolver.update(it, values, null, null)
}
```

Практика: Создание файла (Insert)

ВАЖНО: Никогда не используйте столбец MediaStore.Images.Media.DATA для создания файлов. Он устарел. Используйте RELATIVE_PATH и DISPLAY_NAME.

Практика: Изменение и удаление чужих файлов

Проблема: Ваше приложение не может просто так изменить или удалить файл, который оно не создавало. При попытке это сделать система выбросит RecoverableSecurityException.

Решение: Попросить разрешение у пользователя.

- Этот Exception содержит в себе специальный Intent, который можно запустить.
- Запуск этого Intent покажет пользователю системный диалог: «Разрешить этому приложению изменить этот файл?»
- Если пользователь соглашается, вы получаете право на изменение/удаление только этого одного файла.

Практика: Изменение и удаление чужих файлов

```
try {
    // Пытаемся удалить/изменить чужой файл...
    resolver.delete(fileUri, ...)
} catch (e: SecurityException) {
    // На Android 10 (API 29) это RecoverableSecurityException
    val intentSender = (e as
RecoverableSecurityException).userAction.actionIntent.intentSender
    // Запускаем диалог запроса у пользователя
    startIntentSenderForResult(intentSender, REQUEST_CODE, null, 0, 0, 0,
null)
}
```

Хранение простых данных

Хранение настроек: SharedPreferences

Задача: Сохранить простые данные (настройки, флаги, токены) в формате «ключ-значение».

Классический подход: SharedPreferences

- Плюсы: Просто и быстро для базовых задач.
- Минусы: Синхронный API (может блокировать UI), нет гарантий безопасности при сбоях.

Хранение настроек: SharedPreferences

Пример:

```
val prefs = getSharedPreferences("user_settings", Context.MODE_PRIVATE)
prefs.edit().putInt("user_level", 5).apply()
val level = prefs.getInt("user_level", 0)
```

Key-Value: Классика (SharedPreferences)

Сценарий: Сохранение рекорда в игре.

- Получение доступа к файлу настроек:

```
val prefs = getSharedPreferences(
    "game_settings", // Имя файла
    Context.MODE_PRIVATE
)
```

Key-Value: Классика (SharedPreferences)

Сценарий: Сохранение рекорда в игре.

- Запись значения:
  - Для записи нужен Editor.
  - apply() сохраняет асинхронно, commit() — синхронно. Всегда используйте apply().

```
val newHighScore = 1000
prefs.edit()
    .putInt("HIGH_SCORE", newHighScore)
    .apply()
```

Key-Value: Классика (SharedPreferences)

Сценарий: Сохранение рекорда в игре.

- Чтение значения:
  - При чтении всегда указывайте значение по умолчанию.

```
val highScore = prefs.getInt("HIGH_SCORE", 0) // 0 - значение по умолчанию
Log.d("Game", "Current high score is $highScore")
```

Хранение настроек: SharedPreferences и DataStore

Задача: Сохранить простые данные (настройки, флаги, токены) в формате «ключ-значение».

Современный подход: Jetpack DataStore (Рекомендовано Google)

- Плюсы: Асинхронный (не блокирует UI), использует корутины и Flow, безопасная обработка ошибок.
- Два вида:
  - Preferences DataStore: Для «ключ-значение» (замена SharedPreferences).
  - Proto DataStore: Для хранения типизированных объектов (более сложный, но мощный).

Key-Value: Современный подход (Jetpack DataStore)

Подход: Полностью асинхронный, работает на корутинах и Flow. Не блокирует UI.

1. Добавление зависимости в build.gradle:

```
implementation "androidx.datastore:datastore-preferences:1.1.1"
```

Key-Value: Современный подход (Jetpack DataStore)

Подход: Полностью асинхронный, работает на корутинах и Flow. Не блокирует UI.
2. Создание экземпляра DataStore:

```
// Обычно делается в top-level файле
val Context.dataStore: DataStore<Preferences> by preferencesDataStore(name
= "settings")
val HIGH_SCORE_KEY = intPreferencesKey("high_score")
```

Key-Value: Современный подход (Jetpack DataStore)

Подход: Полностью асинхронный, работает на корутинах и Flow. Не блокирует UI.
3. Запись значения (асинхронно):

```
// Внутри suspend-функции или в scope корутины
suspend fun setHighScore(score: Int) {
    context.dataStore.edit { settings ->
        settings[HIGH_SCORE_KEY] = score
    }
}
```

Key-Value: Современный подход (Jetpack DataStore)

Подход: Полностью асинхронный, работает на корутинах и Flow. Не блокирует UI.
4. Чтение значения (как Flow):

```
// Читаем данные как поток, который будет обновляться при изменениях
val highScoreFlow: Flow<Int> = context.dataStore.data
    .map { preferences ->
        preferences[HIGH_SCORE_KEY] ?: 0
    }
```

Продвинутые сценарии

Шифрование файлов с Jetpack Security

Проблема: Файлы в спецхранилище приватны для других приложений, но на устройстве с root-доступом их все равно можно прочитать. Как защитить содержимое файла от взлома?

Решение: Библиотека androidx.security.crypto.

Как это работает: Библиотека «на лету» шифрует и дешифрует данные при их записи и чтении, используя ключи, надежно сохраненные в Android Keystore.

Шифрование файлов с Jetpack Security

Пример использования:

```
// 1. Создаем зашифрованный файл вместо обычного
val encryptedFile = EncryptedFile.Builder(
    context,
    File(context.filesDir, "secret_data.txt"),
    MasterKeys.getOrCreate(MasterKeys.AES256_GCM_SPEC),
    EncryptedFile.FileEncryptionScheme.AES256_GCM_HKDF_4KB
).build()

// 2. Используем его как обычный файл для записи/чтения
encryptedFile.openFileOutput().use { outputStream ->
    outputStream.write("This is a top secret message!".toByteArray())
}
```

Не-медиа файлы (Storage Access Framework)

Вопрос: А как работать с PDF, ZIP, документами? Для них нет коллекций в MediaStore.

Ответ: Используйте Storage Access Framework (SAF).

Концепция: Ваше приложение не просит разрешений на доступ к хранилищу. Вместо этого оно просит пользователя выбрать файл или папку через системный файловый менеджер.

Не-медиа файлы (Storage Access Framework)

Основные сценарии:

- ACTION_OPEN_DOCUMENT (Открыть файл):
  - Вызываете Intent, пользователь выбирает файл (например, PDF).
  - Вы получаете обратно URI и можете читать его содержимое.
- ACTION_CREATE_DOCUMENT (Сохранить файл):
  - Вызываете Intent, пользователь выбирает папку и имя файла.
  - Вы получаете URI и можете записывать в него данные.
- ACTION_OPEN_DOCUMENT_TREE (Открыть папку):
  - Пользователь выбирает целую папку.
  - Приложение получает доступ ко всем файлам и подпапкам внутри нее.

Важная фишка: Можно запросить постоянный доступ к URI через takePersistableUriPermission(), чтобы не просить пользователя выбирать файл каждый раз при запуске приложения.

Не-медиа файлы (Storage Access Framework)

Сценарий: Приложение-читалка просит пользователя выбрать PDF для открытия.

1. Объявляем ActivityResultLauncher:

```
val openDocumentLauncher = registerForActivityResult(
    ActivityResultContracts.StartActivityForResult()
) { result ->
    // Проверяем, что пользователь успешно выбрал файл
    if (result.resultCode == Activity.RESULT_OK) {
        // requestCode проверять НЕ НУЖНО!
        // Этот код выполнится только для этого лаунчера.
        result.data?.data?.let { uri ->
            readPdfContent(uri)
        }
    }
}
```

Не-медиа файлы (Storage Access Framework)

Сценарий: Приложение-читалка просит пользователя выбрать PDF для открытия.

2. Создаем Intent и запускаем Launcher:

```
fun openFilePicker() {
    val intent = Intent(Intent.ACTION_OPEN_DOCUMENT).apply {
        addCategory(Intent.CATEGORY_OPENABLE)
        type = "application/pdf" // Фильтруем по типу
    }
    openDocumentLauncher.launch(intent)
}
```

Не-медиа файлы (Storage Access Framework)

Сценарий: Текстовый редактор просит пользователя указать, куда сохранить новый .txt файл.

1. Объявляем ActivityResultLauncher:

```
val createDocumentLauncher = registerForActivityResult(
    ActivityResultContracts.StartActivityForResult()
) { result ->
    if (result.resultCode == Activity.RESULT_OK) {
        result.data?.data?.let { uri ->
            writeTextToFile(uri, "Hello, world!")
        }
    }
}
```

Не-медиа файлы (Storage Access Framework)

Сценарий: Текстовый редактор просит пользователя указать, куда сохранить новый .txt файл.

2. Создаем Intent и запускаем Launcher:

```
fun saveFilePicker() {
    val intent = Intent(Intent.ACTION_CREATE_DOCUMENT).apply {
        addCategory(Intent.CATEGORY_OPENABLE)
        type = "text/plain"
        putExtra(Intent.EXTRA_TITLE, "my_document.txt")
    }
    createDocumentLauncher.launch(intent)
}
```

Чтение геолокации из фото (EXIF)

Контекст: По умолчанию Android скрывает из EXIF-метаданных геолокацию (GPS-координаты) из соображений приватности.

Как получить доступ (алгоритм):

1. Запросить разрешение в AndroidManifest.xml:
    - <uses-permission android:name="android.permission.ACCESS_MEDIA_LOCATION" />

(И запросить его у пользователя во время выполнения).

Чтение геолокации из фото (EXIF)

2. При чтении файла запросить оригинал:

```
val photoUri: Uri = /* URI фотографии */
// Для Android 10 (Q) и выше
val uriWithAccess = MediaStore.setRequireOriginal(photoUri)

context.contentResolver.openInputStream(uriWithAccess)?.use { stream ->
    // Читаем EXIF из потока оригинала
    val exifInterface = ExifInterface(stream)
    val latLong = exifInterface.latLong // [latitude, longitude]
}
```

Пакетные операции (Android 11+)

Проблема: Как приложению-галерее удалить 50 фотографий, не показывая пользователю 50 отдельных диалогов подтверждения?

Решение: Запросить у пользователя разрешение на операцию сразу для группы файлов.

Как это работает:

1. Вы собираете URI всех файлов, которые хотите изменить/удалить.
2. Создаете запрос с помощью MediaStore.createWriteRequest() (для изменения) или createDeleteRequest() (для удаления).
3. Запускаете полученный Intent. Система показывает один диалог: "Разрешить приложению X изменить 50 элементов?"
4. В onActivityResult вы получаете ответ и выполняете операцию, если пользователь дал согласие.

Пакетные операции (Android 11+)

Ключевой метод:

```
val urisToModify: List<Uri> = /* ... список URI ... */
val pendingIntent =
MediaStore.createWriteRequest(context.contentResolver, urisToModify)
// Запускаем pendingIntent через startIntentSenderForResult
```

Общие датасеты (BlobStoreManager)

Проблема: У вас есть несколько приложений (например, от одного разработчика), которым нужен один и тот же большой набор данных (например, 1 ГБ карт, словарей или моделей для машинного обучения). Как не заставлять пользователя скачивать этот гигабайт для каждого приложения?

Решение (Android 11+): BlobStoreManager
Позволяет одному приложению «поделиться» большим файлом (BLOB - Binary Large Object) с другими доверенными приложениями.
Система кэширует эти данные, и другие приложения могут получить к ним доступ для чтения, если у них есть разрешение.

Основной сценарий: Экономия места на диске и сетевого трафика для экосистем приложений.

Жизненный цикл: [Приложение А загружает и делится] -> [Приложение Б проверяет и использует].

Общие датасеты (BlobStoreManager)

Жизненный цикл:
[Приложение А загружает и делится] -> [Приложение Б проверяет и использует].

Общие датасеты (BlobStoreManager)

Шаг 1. Приложение А (Владелец) загружает данные и делает их доступными:

```
// 1. Создаем "ручку" (уникальный идентификатор) для наших данных
val blobHandle = BlobHandle.createWithSha256(sha256Hash, "offline_maps",
...)

// 2. Открываем сессию для записи
val sessionId = blobStoreManager.createSession(blobHandle)
val session = blobStoreManager.openSession(sessionId)

// 3. Записываем данные в сессию (например, из сети)
session.openWrite(0, size).use { outputStream -> /* ... write data ... */
}
```

Общие датасеты (BlobStoreManager)

Шаг 1. Приложение А (Владелец) загружает данные и делает их доступными:

```
// 4. Указываем, кто может читать эти данные
session.allowPackageAccess("com.company.courier_app", courierAppCertHash)
session.allowSameSignatureAccess() // Все приложения с нашей подписью

// 5. Публикуем (коммитим) данные
session.commit(executor, callback)
```

Общие датасеты (BlobStoreManager)

Шаг 2. Приложение Б (Потребитель) получает доступ к данным:

```
// 1. Узнаем идентификатор нужных данных (например, с нашего сервера)
val blobHandle = BlobHandle.createWithSha256(sha256Hash, "offline_maps",
...)

// 2. Пытаемся открыть данные по этому идентификатору
try {
    blobStoreManager.openBlob(blobHandle).use { inputStream ->
        // Данные найдены на устройстве! Читаем их.
        /* ... use data ... */
    }
} catch (e: IOException) {
    // Данных на устройстве нет. Запускаем их скачивание...
}
```

Общие датасеты (BlobStoreManager)

Шаг 1. Приложение А (Владелец) загружает данные и делает их доступными:

```
// 4. Указываем, кто может читать эти данные
session.allowPackageAccess("com.company.courier_app", courierAppCertHash)
session.allowSameSignatureAccess() // Все приложения с нашей подписью

// 5. Публикуем (коммитим) данные
session.commit(executor, callback)
```